

TDS Virtual TA

Retrieval-Augmented Question Answering System

Project Report

Course: Tools in Data Science (TDS) — Project 1

Repository: [virtualta-project](#)

License: MIT

1. Executive Summary

This project implements a “Virtual Teaching Assistant” for the TDS course. The system answers student questions automatically by combining a scraped knowledge base of course content with a retrieval-augmented generation (RAG) pipeline powered by a large language model (LLM). It exposes a simple REST API that accepts a natural-language question (optionally with an attached image) and returns a grounded answer together with the source links used to construct it.

The knowledge base is built from two sources: the official TDS course website (crawled and converted to Markdown) and the TDS Knowledge Base category on the course's Discourse forum (downloaded via the Discourse JSON API). Both sources are chunked, embedded, and stored in a local SQLite database, which the API queries at request time using cosine similarity search.

The deliverable is designed to satisfy the course's “Project 1: Virtual TA” assignment, which requires students to build and deploy an API that can be evaluated automatically against a rubric of sample questions (see `promptfoo` configuration, `x.yaml`).

2. Objectives

- **Automate TA support:** Reduce the burden on human teaching assistants by automatically answering common student questions using course materials.
- **Build a knowledge base:** Aggregate and structure content from the course website and Discourse discussion forum into a searchable format.
- **Implement RAG:** Retrieve the most relevant chunks of content for a given question and use an LLM to synthesize a grounded, cited answer.
- **Support multimodal queries:** Allow students to attach a screenshot/image alongside their question for additional context.
- **Expose a REST API:** Provide a simple, well-documented HTTP endpoint (`/query`) that can be evaluated automatically.

3. Data Sources

Two independent scrapers populate the knowledge base:

3.1 Course Website (tds.s-anand.net)

`website_downloader_full.py` uses Playwright to drive a headless Chromium browser, recursively crawling every internal link starting from the 2025-01 course homepage. Each page's main article content is extracted and converted from HTML to Markdown (via `markdownify`), then saved as an individual `.md` file with YAML front-matter (`title`, `original_url`, `downloaded_at`) in the `tds_pages_md/` directory. A consolidated `metadata.json` file records the title, filename, source URL and download timestamp for every page — 382 page entries were captured in the sample database, including repeated sub-section anchors (e.g. “`#!/prompt-engineering?id=...`”) for granular linking.

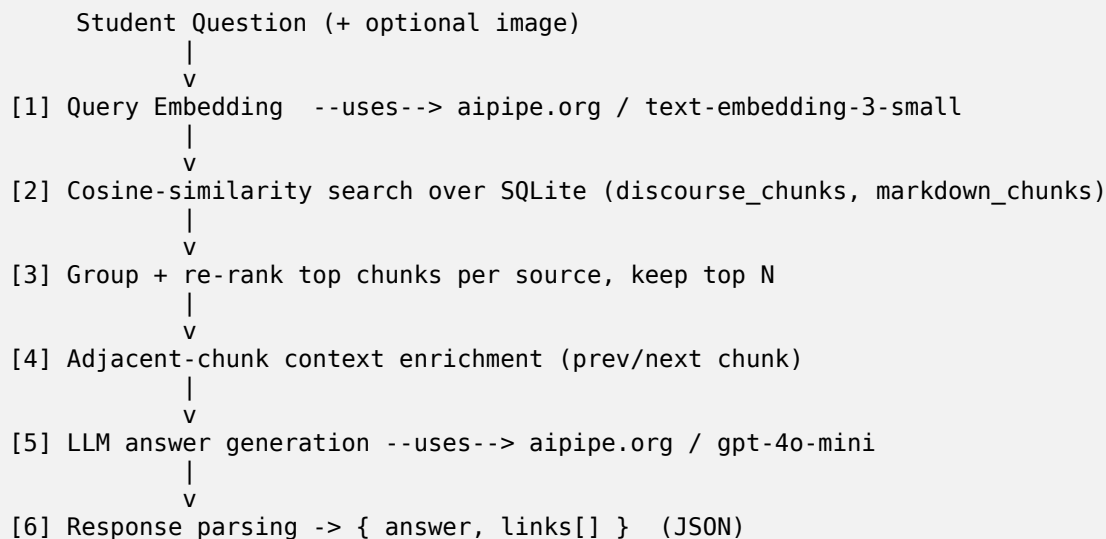
3.2 Discourse Forum Knowledge Base

discourse_downloader_full.py authenticates to the IITM Discourse instance using session cookies and calls the Discourse JSON API to enumerate topics in the “courses/tds-kb” category (ID 34) within a configurable date range. For each topic it downloads the full post stream as JSON, storing per-topic files in discourse_json/ as well as a consolidated discourse_posts.json. The sample database contains 4,085 chunked Discourse posts.

A README.md in the repository explicitly warns that this scraped data carries no guarantee of completeness or correctness and should be validated before production use.

4. System Architecture

The application follows a classic RAG (Retrieval-Augmented Generation) pattern, implemented end-to-end in a single FastAPI service (app.py):



Key design characteristics:

- **Storage:** A single SQLite file (knowledge_base.db, ~92 MB in the shipped sample) with two tables — discourse_chunks and markdown_chunks — each storing chunk text alongside a JSON-serialized embedding vector.
- **Embeddings:** Generated via OpenAI's text-embedding-3-small model, accessed through the “aipipe” proxy (aipipe.org/openai/v1/embeddings) rather than calling OpenAI directly, with automatic retry/back-off on rate limits (HTTP 429).
- **Similarity search:** Performed in Python/NumPy at query time (cosine_similarity) with a similarity threshold of 0.50 and a cap of 15 final results (MAX_RESULTS), grouped by source document so a single page/post cannot dominate the context window.
- **Context enrichment:** For every matched chunk, the immediately preceding and following chunks from the same document are appended, giving the LLM more surrounding context.

- **Answer generation:** gpt-4o-mini (again via the aipipe proxy) is prompted to answer strictly from the retrieved context and to return a structured answer plus a “Sources:” section, which is then parsed with regular expressions into a clean { answer, links[] } JSON payload.
- **Multimodal support:** If an image (base64) is supplied with the question, it is first sent to a vision-capable model to obtain a textual description, which is merged with the question text before embedding — falling back gracefully to text-only search if the vision call fails.
- **Frontend:** A minimal static HTML page (static/index.html) is served at the root path for manual testing of the API.

5. API Reference

Endpoint	Method	Purpose
/	GET	Serves the static demo front-end (static/index.html).
/query	POST	Main RAG endpoint. Accepts { question, image? } and returns { answer, links[] }.
/health	GET	Reports DB connectivity, API-key status, and chunk/embedding counts for both tables.

Example request body for /query:

```
POST /query
Content-Type: application/json

{
  "question": "Should I use Docker or Podman for this course?",
  "image": null
}
```

Example response shape:

```
{
  "answer": "Podman is recommended, but Docker is also acceptable...",
  "links": [
    { "url": "https://tds.s-anand.net/#/docker", "text": "Containers: Docker, Podman" }
  ]
}
```

6. Technology Stack

Layer	Technology
Web framework	FastAPI + Uvicorn
Data validation	Pydantic models (QueryRequest, QueryResponse, LinkInfo)
Database	SQLite (built-in, file-based)
Embeddings & LLM	OpenAI text-embedding-3-small and gpt-4o-mini, via the aipipe.org proxy
Web scraping	Playwright (JS-rendered site), Requests (Discourse JSON API)
HTML → Markdown	markdownify, html2text
Numerical computation	NumPy (cosine similarity)
Config / secrets	python-dotenv (.env file, API_KEY variable)

7. Evaluation

The project includes a promptfoo configuration (x.yaml) that automates grading of the deployed API against a rubric of sample questions, mirroring how the course evaluators assess submissions. It:

- **Posts questions** to the local /query endpoint (or a deployed equivalent) with an optional attached image.
- **Validates the JSON schema** of every response, requiring an answer string and a links array of { url, text } objects.
- **Applies llm-rubric assertions** that use gpt-4o-mini as a judge to check whether the answer satisfies specific factual criteria (e.g. correctly recommending Podman over Docker, or acknowledging missing knowledge for out-of-scope questions such as exam dates).
- **Checks for exact source URLs** with contains assertions, ensuring the returned links reference the correct underlying discussion thread or documentation page.

This automated harness allows the system's accuracy and groundedness to be re-tested quickly after any change to the retrieval pipeline or knowledge base.

8. Limitations and Risks

- **Data completeness/correctness:** The README explicitly disclaims any guarantee that the scraped data is complete or accurate — answers are only as good as the underlying scrape.
- **Scraper fragility:** The Discourse downloader relies on hard-coded session cookies, which expire and require manual renewal; the website downloader depends on the site's DOM structure (article.markdown-section#main selector) and will break if the site is redesigned.

- **Static knowledge cutoff:** The knowledge base only contains data captured up to the last scrape (the shipped copy covers Jan–Apr 2025 Discourse posts and the 2025-01 website snapshot), so it cannot answer questions about events after that date — by design, the system should say it doesn't know rather than hallucinate.
- **In-memory similarity search:** Cosine similarity is computed by iterating over every embedding row in Python at query time rather than using a dedicated vector index (e.g. FAISS, pgvector), which will not scale well as the knowledge base grows.
- **Secrets in source:** The Discourse downloader script contains a real (now-rotated) session cookie hard-coded as an example, and the API key is expected via a .env file that must not be committed.
- **Single external dependency:** Both embeddings and answer generation depend on the third-party aipipe.org proxy being available; there is no fallback provider.

9. Possible Future Improvements

- Migrate similarity search to a dedicated vector database (e.g. LanceDB, ChromaDB, or SQLite's vector extensions) for better scalability and query latency.
- Automate credential refresh for the Discourse scraper (e.g. OAuth or an API token) instead of manually copied session cookies.
- Add caching of embeddings/answers for repeated or similar questions to reduce API cost and latency.
- Expand automated evaluation coverage (x.yaml) with a broader and continuously updated set of rubric questions.
- Containerize the application (Dockerfile) for reproducible deployment, referenced implicitly by the course's own Docker/Podman guidance.

10. Conclusion

The TDS Virtual TA project delivers a functional, end-to-end retrieval-augmented question-answering system tailored to a specific course's content. By combining scraped website documentation and forum discussions into a unified, embedded knowledge base, and pairing it with an LLM-based answer synthesis step, the system can automatically field a wide range of student questions with cited, source-grounded answers — while explicitly declining to answer when the necessary information is not present in its knowledge base. The accompanying promptfoo evaluation harness provides a repeatable way to verify the quality of the system as it evolves.